



TARDIS

< PREDICTING THE UNPREDICTABLE />



TARDIS

Preliminaries



files to deliver: tardis_eda.ipynb, tardis_model.ipynb, tardis_dashboard.py,
language: Python with pandas, numpy, matplotlib, seaborn, scikit-learn, streamlit
coding style: Your project must be formatted with [ruff formatter](#)

You are part of a newly formed **SNCF Data Analysis Service**, dedicated to improving the efficiency of train travel across the country. Your mission? Analyze historical train delay data, uncover hidden patterns, and develop a predictive model that can forecast delays before they happen.

The SNCF has entrusted your team with making the railway system more efficient and transparent. If you succeed, your dashboard will be used by thousands of travelers to better plan their journeys. If you fail... well, expect a lot more unhappy commuters. No pressure!

Using the provided dataset (available along with the subject on the intranet), your job is to clean and analyze historical delay data, develop a simple predictive model, and present your insights through an interactive Streamlit dashboard.



Objectives

- ✓ Data Cleaning & Preprocessing – Handle missing values, inconsistencies, and prepare the dataset for analysis.
- ✓ Exploratory Data Analysis (EDA) – Generate insightful visualizations to understand trends and correlations.
- ✓ Predictive Modeling – Implement a basic machine learning model to predict train delays.
- ✓ Dashboard Development – Create an interactive web app using Streamlit to display insights and allow user interaction.



Step 1: Data Exploration & Cleaning

Before diving into analysis, ensure the dataset is clean and structured. Start by inspecting its structure and identifying any missing values or duplicate entries. Convert data types where necessary and create additional features, such as day of the week or peak hours, to enhance predictions.

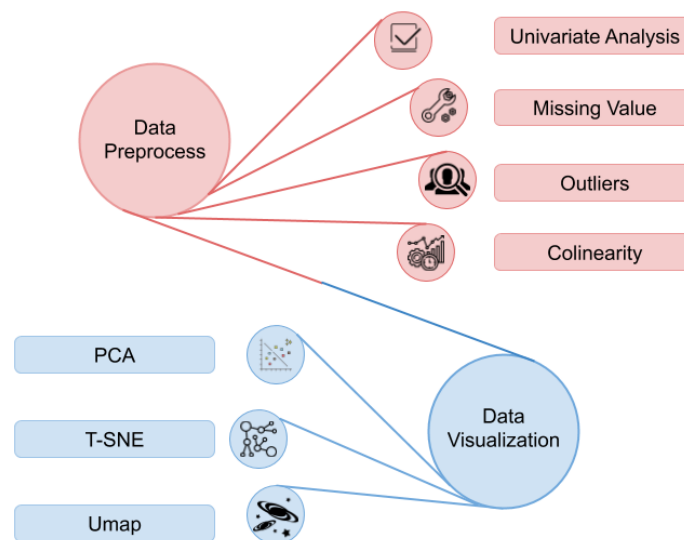
Tasks:

- ✓ Load and inspect the dataset (will be named `dataset.csv` during automated tests).
- ✓ Handle missing values and remove duplicate entries.
- ✓ Convert columns to appropriate data types.
- ✓ Perform feature engineering to create new useful variables.

Expected Deliverable:

A `tardis_eda.ipynb` Jupyter Notebook containing:

- Data loading and initial inspection
- Missing values and duplicates handling
- Data type conversions
- Feature engineering (temporal features, delay categories, etc.)
- Output: `cleaned_dataset.csv` file



Step 2: Data Visualization & Analysis

Once the dataset is clean, use visualizations to understand trends and patterns in train delays. This will help you uncover insights that may improve predictive modeling

Tasks:

- ✓ Generate summary statistics to understand the dataset better.
- ✓ Plot delay distributions and identify common delay durations.
- ✓ Compare delays across different stations and times of day.

Expected Deliverable:

Continue in `tardis_eda.ipynb` with:

- Descriptive statistics for key variables
- Multiple visualizations (distributions, comparisons, correlations)
- Written interpretations and insights



Step 3: Building a Prediction Model

Build a **regression model** to predict train delay duration in minutes. Your model should take journey characteristics as input and output an estimated delay.

Prediction Target:

- ✓ **What to predict:** Delay duration (continuous value in minutes)
- ✓ **Input features examples:** Departure station, arrival station, departure time, day of week, train type
- ✓ **Minimum requirement:** Model must perform better than a baseline (e.g., predicting mean delay)

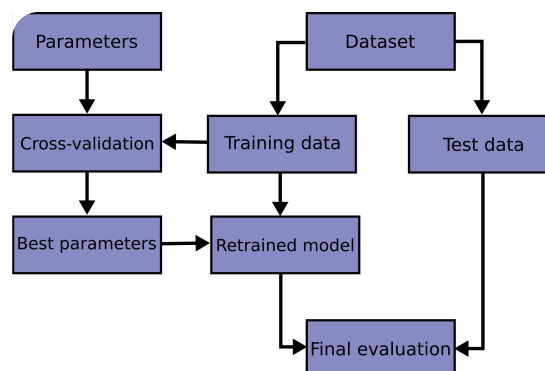
Tasks:

- ✓ **Feature Engineering:** Encode categorical variables and create temporal features (hour, day of week, peak hours)
- ✓ **Model Training:** Train at least 2-3 regression models (linear and tree-based algorithms)
- ✓ **Evaluation:** Use appropriate regression metrics (RMSE, MAE, R^2) to compare models
- ✓ **Model Selection:** Select the best performing model and apply hyperparameter tuning
- ✓ **Justification:** Document your choices and the improvements gained

Expected Deliverable:

A `tardis_model.ipynb` Jupyter Notebook containing:

- Feature engineering and selection process
- Multiple models trained and compared with metrics
- Hyperparameter tuning results
- Final model selection justification
- Saved model file for dashboard integration



Step 4: Developing the Streamlit Dashboard

Make your analysis accessible with an interactive dashboard. The dashboard should allow users to explore key insights and interact with the predictive model.

Must have (mandatory)

- ✓ **Delay distribution visualizations:** At least one chart showing delay patterns
- ✓ **Summary statistics panel:** Display key metrics (average delays, total trips, punctuality rate)
- ✓ **Prediction interface:** User inputs for journey parameters + predict action + clear output showing predicted delay

Should have

- ✓ **Station-level comparative analysis:** Charts comparing delays across different stations
- ✓ **Interactive filters:** User controls to filter by station, route, or time period
- ✓ **Dynamic visualizations:** Charts that update based on user selections
- ✓ **Correlation heatmap:** Visual representation of relationships between delay factors
- ✓ **Model explanation:** Display feature importance or which input features influenced the prediction

Could have

- ✓ **Multiple visualization types:** Combine histograms, line charts, scatter plots, and heatmaps
- ✓ **Advanced interactivity:** Date range pickers, multi-select filters, drill-down capabilities
- ✓ **Prediction confidence indicators:** Show model uncertainty or prediction intervals
- ✓ **Comparative predictions:** Allow users to compare predictions for different scenarios
- ✓ **Data quality indicators:** Show data completeness or reliability metrics
- ✓ **Export functionality:** Allow users to download filtered data or charts

Expected Deliverable:

A `tardis_dashboard.py` Python script with:

- Working Streamlit application
- All "Must Have" features implemented
- Model integration for predictions
- README.md with setup and usage instructions



Keep the layout simple and organized – better fewer features done well than many done poorly

Final Deliverables Summary

Your repository must contain the following files:

File	Description
requirements.txt	All project dependencies
tardis_eda.ipynb	Data cleaning, exploration, and feature engineering
cleaned_dataset.csv	Processed dataset output from EDA notebook
tardis_model.ipynb	Model training, evaluation, and selection
model.pkl OR model.joblib	Trained model file
tardis_dashboard.py	Interactive Streamlit dashboard
README.md	Installation, usage, and project documentation

Bonus Challenges:

- ✓ Implement a feature selection technique to optimize the model.
- ✓ Add real-time data updates from SNCF open data.
- ✓ Use advanced visualization techniques like animated charts or geospatial maps.
- ✓ Experiment with deep learning models for improved accuracy.
- ✓ Include an explanatory model component to provide reasons for predictions.

v 1.1

{EPITECH}